

Nama : Sudrajat
Kelas : TI-24-PA2
NPM : 242310013

Praktikum 4

Langkah 1: Instalasi dan Persiapan Lingkungan

Penjelasan:

Pada langkah ini, dilakukan instalasi Node.js (versi LTS) sebagai runtime environment. Setelah proses instalasi selesai, verifikasi dilakukan melalui terminal menggunakan perintah ``node -v`` dan ``npm -v`` untuk memastikan bahwa Node.js dan Node Package Manager telah terpasang dengan benar di sistem operasi.

Langkah 2: Membangun Backend Server

Penjelasan:

Direktori baru bernama "backend" dibuat. Di dalam direktori tersebut, dilakukan inisialisasi project Node menggunakan ``npm init -y``. Kemudian, beberapa pustaka esensial diinstal menggunakan perintah ``npm install express mysql2 cors``. File ``index.js`` dikonfigurasi sebagai entry point server Express sederhana yang berjalan pada port awal (3000).

Langkah 3: Integrasi Database MySQL

Penjelasan:

Database bernama ``pw_praktikum_db`` dibuat pada sistem MySQL lokal. Selanjutnya, kode pada file ``index.js`` dimodifikasi dengan menambahkan fungsi ``mysql.createConnection()`` untuk menyambungkan server backend dengan database MySQL. Terdapat validasi apabila koneksi berhasil atau gagal (misalnya karena kredensial yang salah atau server MySQL belum menyala).

Langkah 4: Membangun Frontend dengan React

Penjelasan:

Project frontend React dibuat menggunakan Next.js melalui perintah ``npx create-next-app@latest my-first-app-react`` dengan konfigurasi menggunakan App Router, TypeScript, dan menonaktifkan Tailwind CSS. File ``src/app/page.tsx`` kemudian diubah untuk merender antarmuka pengguna sederhana yang menampilkan tulisan "Hello World!!!". Aplikasi dijalankan pada port default 3000.

Langkah 5: Integrasi Frontend dan Backend

Penjelasan:

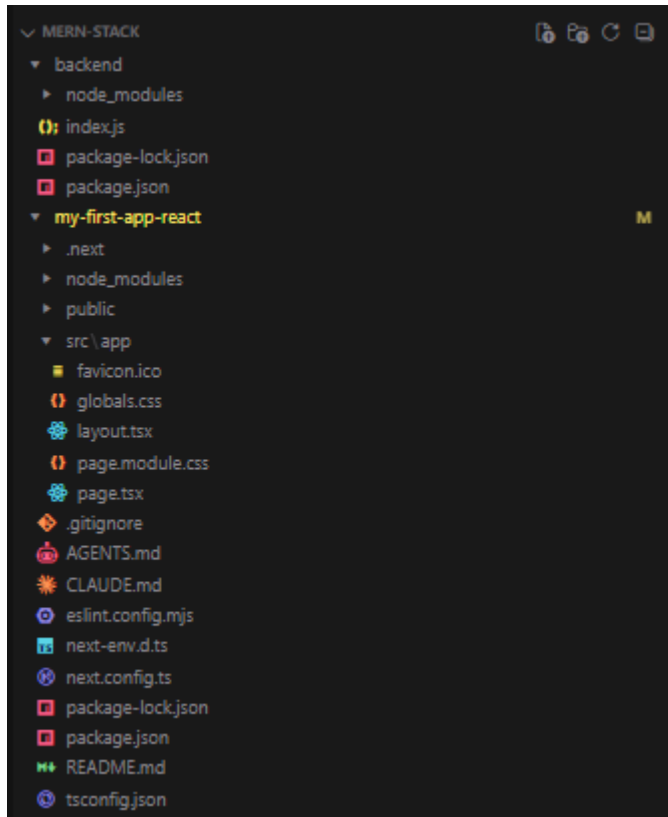
Agar frontend dan backend dapat berjalan bersamaan di localhost, port server backend pada ``index.js`` diubah menjadi 3001. Sebuah endpoint API ``/api/info`` dibuat. Untuk mengatasi kebijakan Same-Origin Policy (SOP), middleware CORS ditambahkan di backend agar memberikan izin akses dari asal ``http://localhost:3000``.

Di sisi frontend, Axios diinstal dengan ``npm install axios``. File ``page.tsx`` dimodifikasi menggunakan hooks ``useState`` dan ``useEffect`` untuk melakukan HTTP GET request ke backend. Untuk mematuhi aturan ketat TypeScript di Next.js, tipe data ``any`` diganti dengan

union type yang aman, dan fungsi request dibungkus di dalam scope yang tepat untuk menghilangkan seluruh error Linter.

Dokumentasi

Struktur File



page.tsx pada file my-first-app-react

```
"use client";

import axios from "axios";
import { useEffect, useState } from "react";

export default function Home() {
  const [info, setInfo] = useState<Record<string, unknown> | string | null>(
    null,
  );

  useEffect(() => {
    const GetAPIInfo = async () => {
      try {
        const response = await axios.get("http://localhost:3001/api/info");
        setInfo(response.data);
      } catch (error) {
        console.error("Error fetching API data:", error);

        if (axios.isAxiosError(error)) {
```

```

        setInfo(error.message);
    } else if (error instanceof Error) {
        setInfo(error.message);
    } else {
        setInfo("Terjadi kesalahan yang tidak diketahui");
    }
    }
};

GetAPIInfo();
}, []);

return (
    <div>
        <h1>Hello World!!</h1>
        {info && (
            <pre
                style={{
                    backgroundColor: "#f4f4f4",
                    padding: "15px",
                    borderRadius: "8px",
                    border: "1px solid #ddd",
                    overflow: "auto",
                }}
            >
                {JSON.stringify(info, null, 2)}
            </pre>
        )}
    </div>
);
}

```

index.js pada file backend

```

const express = require("express");
const mysql = require("mysql2");
const cors = require("cors");
const app = express();

app.use(
    cors({
        origin: "http://localhost:3000",
        credentials: true,
    })
);

app.use(express.json());

```

```
const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "pw_praktikum_db",
});

db.connect((err) => {
  if (err) {
    console.error("Koneksi ke database gagal:", err.message);
    process.exit(1);
  }
  console.log("Koneksi ke database MySQL berhasil!");
});

app.get("/", (req, res) => {
  res.send("Server berjalan");
});

app.listen(3001, () => {
  console.log("Server running on port 3001");
});

app.get("/api/info", (req, res) => {
  res.json({
    message: "API MERN Stack Build by Express JS",
    version: "1.0.0",
    status: "active",
    timestamp: new Date(),
  });
});
```

Backend berjalan

```
C:\Windows\System32\cmd.exe - node index
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

D:\Kulyeah\Semester 4\Pemrograman Web\MERN-STACK\backend>node index
Server running on port 3001
Koneksi ke database MySQL berhasil!
^C
D:\Kulyeah\Semester 4\Pemrograman Web\MERN-STACK\backend>node index
Server running on port 3001
Koneksi ke database MySQL berhasil!
```

Frontend berjalan

```
next-server (v16.2.11)
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

D:\Kulyeah\Semester 4\Pemrograman Web\MERN-STACK\my-first-app-react>npm run dev

> my-first-app-react@0.1.0 dev
> next dev

▲ Next.js 16.2.11 (Turbopack)
- Local:      http://localhost:3000
- Network:    http://192.168.56.1:3000
■ Ready in 1462ms

o Compiling / ...
  GET / 200 in 7.2s (next.js: 6.8s, application-code: 400ms)
  ./src/app/page.tsx:15:1
Module not found: Can't resolve 'axios'
   13 |   "use client";
   14 |
>  15 |   import axios from "axios";
      |         ^^^^^^^^^^^^^^^^^^^^^
   16 |   import { useEffect, useState } from "react";
   17 |
   18 |   export default function Home() {

https://nextjs.org/docs/messages/module-not-found
```